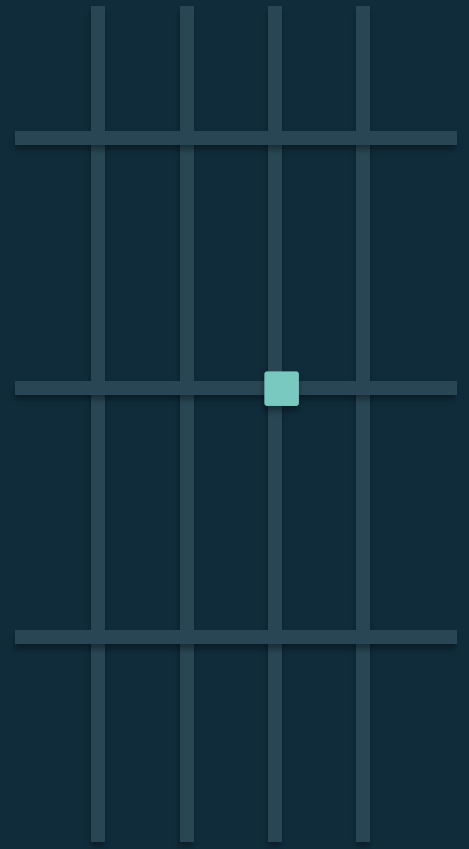
Verkehrsunfälle in Zürich

Von offenen Daten zu nachvollziehbaren Antworten.

Eine reproduzierbare Python-/SQL-Pipeline für registrierte Unfälle von 2011–2025.



70’767
registrierte Unfälle

15
Unfalljahre

3
Forschungsfragen

Ruben Souto · Issa Fawaz · Drin Muslija

Drei Fragen verbinden Technik und Analyse

Ausgangslage: Unfälle verursachen Schäden. Offene Daten machen Muster untersuchbar.

F1 Wann?

Tageszeit und Wochentag

Stundenanteile vergleichen;
H1 prüft Unterschiede.

F2 Mit welcher Beteiligung?

Beteiligung und Schwere
bei Personenschadenunfällen

Drei Vergleiche; H2.

F3 Was verändert sich?

Anzahl und Zusammensetzung
über die Jahre

Jahresreihen und Anteile.

Engineering-Ziel: dieselbe Quelle mit denselben Regeln nachvollziehbar auswerten.

Eine Zeile entspricht einem registrierten Unfall

Open Data Zürich · Datenlieferant: Bundesamt für Strassen ASTRA

Was ist enthalten?

2011–2025 · 36 Rohspalten
Zeit, Schwere, Unfallart
Beteiligung und Koordinaten

70’767 eindeutige Unfall-IDs

Bruch in der Erfassung

Ab Juli 2015 werden auch Bagatellunfälle gemeldet.

Gesamtzahlen über diesen Bruch vorsichtig vergleichen.

Unklare Definition

Beteiligung ist im Katalog auf Personenschäden bezogen; positive Werte auch bei Sachschäden vorhanden.
→ F2 nur Personenschäden.

Keine Daten zum Verkehrsaufkommen → keine Unfallrisiken pro Fahrt oder Kilometer.

Daten: Open Data Zürich / Bundesamt für Strassen ASTRA · Snapshot 13.10.2026

Eine lokale Batch-Pipeline mit klarer Herkunft

Generation liegt ausserhalb unseres Projekts; die eigene Verarbeitung beginnt beim Bezug.

01
Generation
Polizei / ASTRA
Erfassung

›

02
Ingestion
CSV + Metadaten
Snapshot + Hash

›

03
Storage
Raw-Dateien erhalten
CSV und SQLite

›

04
Transformation
Profil, Typen, Features
Qualitätsbericht

›

05
Serving
SQL / Notebooks
Diagramme und Folien

Unterströmungen: Sicherheit · Governance · DataOps · Orchestrierung · Software Engineering

Storage begleitet mehrere Phasen; die Reihenfolge ist kein starres Wasserfallmodell.

Ein Snapshot schützt vor unbemerkten Datenwechseln

Standardlauf offline · Ein neuer Download ist eine bewusste Entscheidung.

1 Beschaffen und festhalten

Öffentliche CSV und Katalogmetadaten

Abrufzeit, Quelle und SHA-256 speichern

Rohdaten unverändert aufbewahren

Neue Downloads als separaten Snapshot

2 Vor der Verarbeitung prüfen

Datei und Metadaten vorhanden?

Dateigrösse und Prüfsumme passend?

CSV-Struktur und Pflichtfelder gültig?

Bei Abweichung sichtbar abbrechen.

SHA-256 bestätigt Dateiintegrität – nicht die fachliche Richtigkeit der Unfalldaten.

Nachweis: ingestion.py · data/raw/source_metadata.json

SQLite passt zu unserem tatsächlichen Arbeitsumfang

Lokale Analyse · ein kontrollierter Schreibprozess · überwiegend lesende Abfragen

CSV

Einfacher Austausch
Originalformat der Quelle

Keine Datenbankregeln
→ Raw und Export

SQLite · gewählt

SQL in einer portablen Datei
Primärschlüssel + CHECKs
Transaktionaler Aufbau

Kein Serverbetrieb nötig

PostgreSQL

Zentraler Mehrnutzerbetrieb
Viele parallele Schreibzugriffe

Dafür wären Server und
Betrieb zusätzlich nötig.

Relational: feste Spalten, eindeutige Unfall-ID und SQL-Aggregationen für F1–F3.

Begründung: sqlite.org/whentouse.html · Schema: src/de_projekt/schema.sql

Bereinigen heisst auch: begründet nichts löschen

Profilbefund: keine leeren Felder, geprüften NA-Marker oder Duplikate im Snapshot.

Von 36 auf 22 Spalten

19 Originalfelder + 3 neue Merkmale

Explizite Zahlen-, Text- und boolesche Typen

Sprachredundanzen nur im Export entfernen

Keine Zeilen löschen, nichts imputieren

Unbekannt bleibt NULL; ungültig → Abbruch

Drei nachvollziehbare Features

is_weekend: Samstag / Sonntag

time_period: vier Sechs-Stunden-Blöcke

vulnerable_road_user: Fussgänger ODER Velo

H1 nutzt Wochenende × Originalstunde;

die anderen Features ergänzen den Bestand.

70’767 hinein → 70’767 hinaus · Regeln und Prüfsummen stehen im Qualitätsbericht.

Zehnmal mehr Daten brauchen rund zehnmal so lange

Gemessen: echter Snapshot vervielfacht, unveränderte Pipeline · Storage ≈ 70 % der Zeit

Faktor	Unfälle	Transformation	Storage	Total	Arbeitsspeicher
1×	70’767	1,4 s	3,5 s	5,2 s	304 MB
2×	141’534	2,9 s	6,2 s	9,6 s	440 MB
5×	353’835	7,1 s	18,9 s	27,1 s	966 MB
10×	707’670	14,1 s	39,9 s	56,2 s	1’835 MB

Grenze ist der Arbeitsspeicher: Mit 8 GB reicht der Entwurf für das 30- bis 40-Fache.

Nachweis: src/de_projekt/benchmark.py · data/processed/scaling_benchmark.csv · macOS (Apple Silicon), Python 3.14.7, 26.10.2026

Wir testen Zusammenhänge und berichten ihre Stärke

Pearson-Chi-Quadrat · α = 0,05 · Cramérs V ergänzt den p-Wert

H1 · Zeitliche Verteilung

H0: Gleiche Stundenverteilung an Werktagen und Wochenenden.

Alternative: unterschiedliche Verteilung.

Tabelle: 2 Gruppen × 24 Stunden

Alle 70’767 Unfälle

H2 · Beteiligung und Schwere

H0: Beteiligung und Schwere unabhängig.

Alternative: ein Zusammenhang besteht.

Drei Tests: Fussgänger, Velo, Motorrad

Je 2 × 3 Zellen · 18’164 Personenschäden

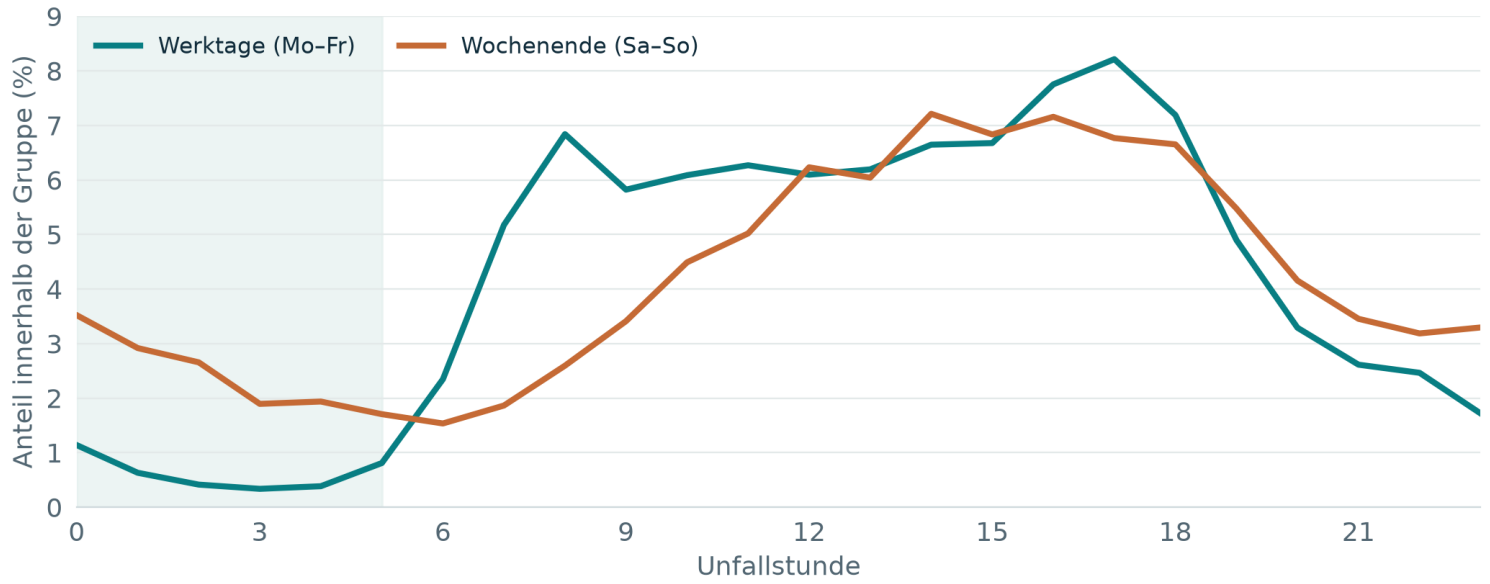
Holm korrigiert die drei p-Werte.

Erwartete Zellen alle > 5; zeitliche Unabhängigkeit bleibt eine Modellannahme.

Methode: SciPy chi2_contingency · Holm (1979), Scandinavian Journal of Statistics 6(2), 65-70

Am Wochenende verlagert sich die Stundenverteilung

Relative Anteile innerhalb jeder Gruppe; kein blosser Vergleich von fünf mit zwei Tagen.



Nacht · 00–05 Uhr

3,71 % werktags

14,63 % am Wochenende

Spitzenstunde: 17 / 14 Uhr

$p < 0{,}001$ · $V = 0{,}234$

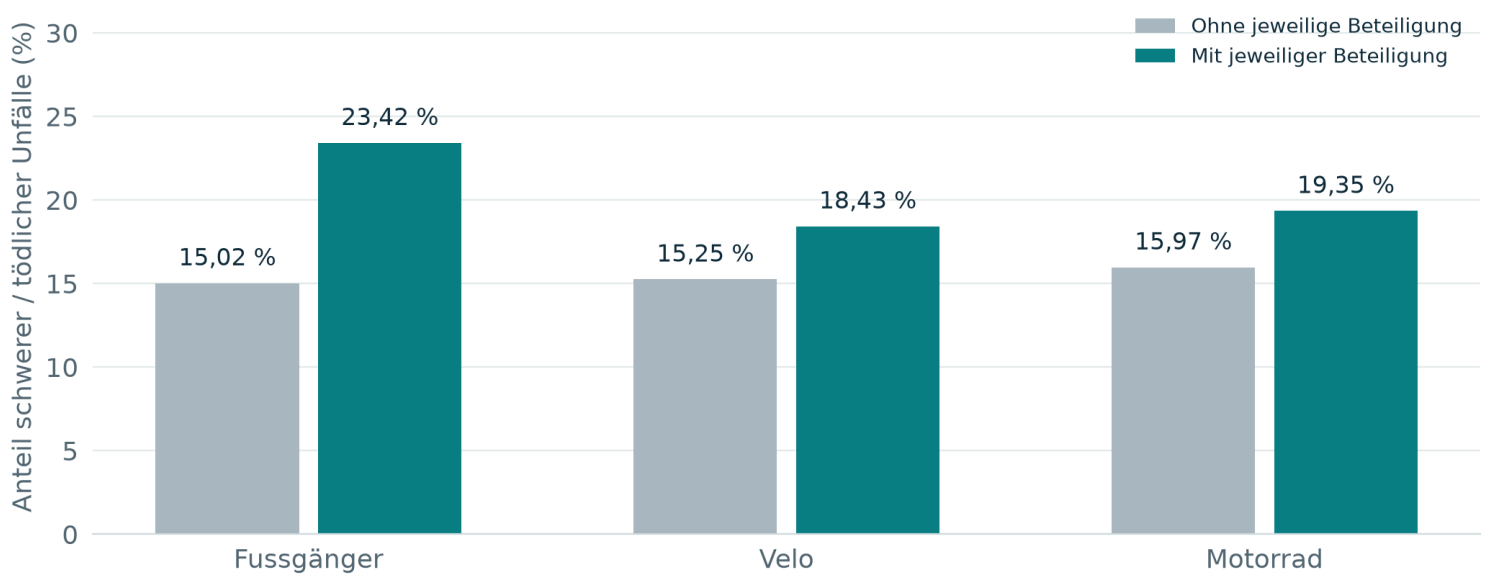
H0 wird verworfen.

Befund: andere Verteilung registrierter Unfälle; kein höheres Risiko pro Wochenendfahrt.

Daten: Open Data Zürich / Bundesamt für Strassen ASTRA · Snapshot 13.10.2026

Beteiligung hängt mit der Unfallschwere zusammen

Nur Personenschadenunfälle · Gezeigt: schwer / tödlich zur verständlichen Einordnung.



Alle drei signifikant

Nach Holm-Korrektur

Cramérs V:

Fussgänger 0,104

Velo 0,054

Motorrad 0,036

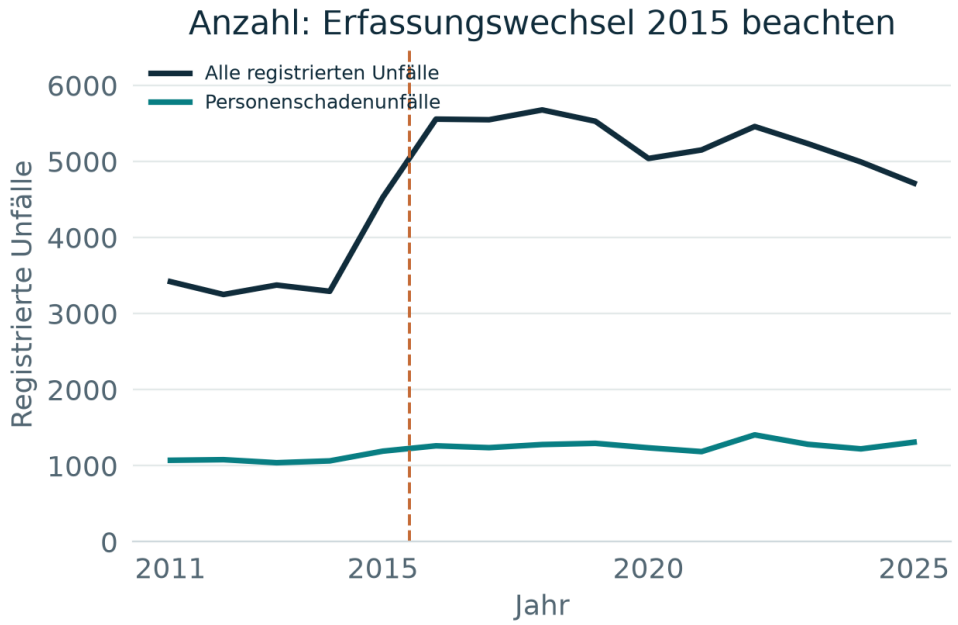
→ Kleine Assoziationen

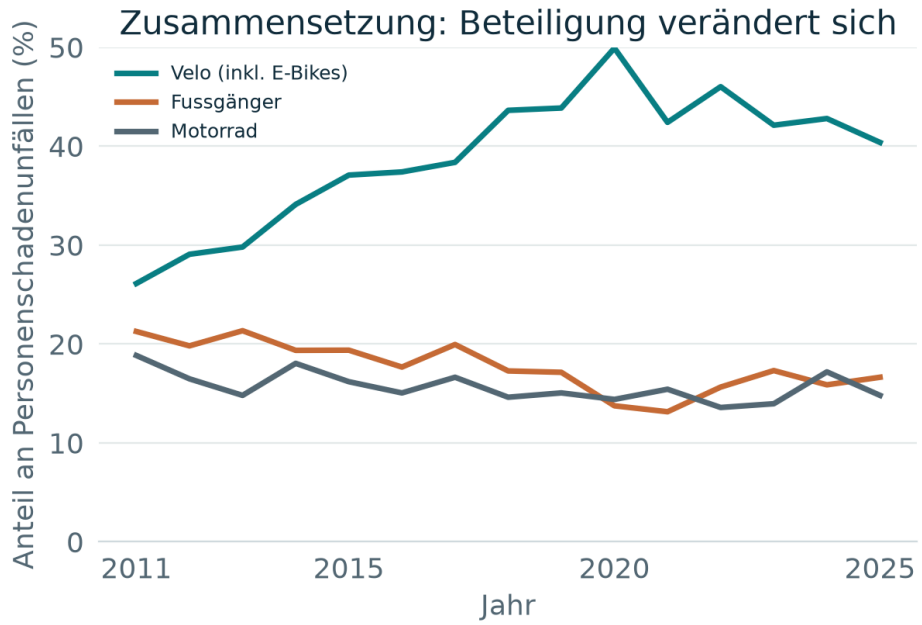
Der Test nutzt drei Schweregrade. Beteiligung sagt nicht, welche Person verletzt wurde.

Daten: Open Data Zürich / Bundesamt für Strassen ASTRA · Snapshot 13.10.2026

Anzahl und Zusammensetzung verändern sich

Eine Zeitreihe zeigt mehr als der Vergleich des ersten mit dem letzten Jahr.





2016 → 2025: total 5’556 → 4’714 · Personenschäden 1’262 → 1’310 · keine stetige Zunahme.

Daten: Open Data Zürich / Bundesamt für Strassen ASTRA · Snapshot 13.10.2026

Technisch geprüft, fachlich bewusst begrenzt

Reproduzierbarkeit sichert den Rechenweg. Sie beseitigt keine Grenzen der Quelle.

Was ist abgesichert?

44 Tests und Codeprüfung erfolgreich

Pipeline + beide Notebooks erneut geprüft

Ergebnisse im isolierten Lauf reproduziert

Prüfsummen und SQLite-Integrität geprüft

Fehlerfälle: ungültige Werte, IDs, Metadaten

Was bleibt offen?

Dunkelziffer nicht registrierter Unfälle

Keine Verkehrsleistung / Expositionsdaten

Keine kausale oder adjustierte Analyse

Mögliche zeitliche Abhängigkeiten

Erfassungsbruch und Definitionsdiskrepanz

Nachvollziehbare Schlussfolgerungen statt unbelegter Risiko- oder Ursachenbehauptungen.

Drei Antworten auf einer nachvollziehbaren Grundlage

Der Mehrwert liegt in fachlich eingeordneten Ergebnissen und einer überprüfbaren Verarbeitung.

F1 **Zeitmuster unterscheiden sich.**

Am Wochenende liegt ein deutlich höherer Anteil der registrierten Unfälle in der Nacht.

F2 **Beteiligung und Schwere sind assoziiert.**

Alle drei Vergleiche sind signifikant; kleine Effektgrössen, keine Kausalaussage.

F3 **Der Bestand verändert sich uneinheitlich.**

Gesamtzahl und Personenschäden entwickeln sich unterschiedlich; der Veloanteil nimmt zu.

Teil 1: Datengrundlage, initiale Transformation und Analyse · kein Machine Learning.

Vom Ergebnis zurück zur Quelle – jede Stufe ist vorhanden

Verstehen

README

Konzept und Entscheidungen

Ausgeführte Notebooks

Nachvollziehen

Python-Code und SQL-Schema

Raw + Metadaten

CSV + SQLite + Berichte

Prüfen

44 Tests und Codeprüfung

Dokumentierte Ausführung

Git-Branch: abgabe/teil1

Fragen?

Teil 2: Erweiterungen erst auf separatem Arbeitsstand.

Eine Unfalltabelle genügt für unsere Fragen

accidents: 70’767 Zeilen · 22 Spalten · AccidentUID als Primärschlüssel

Datenmodell

Zeit und Schwere

Unfall- und Strassenart

Beteiligungsmerkmale

Koordinaten + 3 Features

pipeline_metadata:

Herkunft und Regelversionen

```
SELECT AccidentYear, COUNT(*) AS n
FROM accidents
WHERE AccidentSeverityCategory
      IN ('as1', 'as2', 'as3')
GROUP BY AccidentYear
ORDER BY AccidentYear;
```

STRICT-Typen · CHECK-Regeln · transaktionaler Aufbau · Integritäts- und Wertevergleich

Signifikanz, Effektgrösse und Fallzahl gemeinsam lesen

Pearson-Chi-Quadrat ohne Kontinuitätskorrektur · H1 separat, H2 als Familie korrigiert

Test	n	χ^2	df	p / p(Holm)	Cramérs V
H1 · Zeit	70’767	3’877,063	23	< 0,001	0,234
H2 · Fussgänger	18’164	194,879	2	$1{,}444 \times 10^{-42}$	0,104
H2 · Velo	18’164	52,602	2	$7{,}563 \times 10^{-12}$	0,054
H2 · Motorrad	18’164	23,961	2	$6{,}267 \times 10^{-6}$	0,036

Kleinste erwartete Zelle: H1 114,61 · H2 mindestens 15,59. H1-p-Wert numerisch unterlaufen.

Nachweis: data/processed/analysis_summary.json · hypothesis_tests_h2.csv · analysis.py

Qualität und Verantwortung begleiten jede Phase

Eine einfache Architektur kann klare Kontrollen besitzen und später erweitert werden.

Sicherheit & Governance

Öffentliche Daten, keine Secrets

Quelle, Lizenz und Snapshot dokumentiert

DataOps & Qualität

Tests, Prüfsummen und Qualitätsberichte

Sichtbarer Abbruch bei Fehlern

Orchestrierung & Engineering

Getrennte Module; explizite Pfadübergaben

Git-Versionierung und reproduzierbarer Lauf

Architektur & Skalierbarkeit

Bei grösseren Daten: chunkweise Verarbeitung

Bei zentralem Mehrnutzerbedarf: PostgreSQL

Skalierung ist ein begründeter Erweiterungsweg; keine bereits getestete Serverarchitektur.

Quellen, Regeln und Unterstützung offenlegen

Die fachliche Verantwortung und die mündliche Erklärung liegen beim Projektteam.

Daten	Open Data Zürich · Bundesamt für Strassen ASTRA Snapshot vom 13.10.2026; Lizenz laut Katalog: Creative Commons Attribution
Datenbank	SQLite: Appropriate Uses For SQLite https://www.sqlite.org/whentouse.html
Statistik	SciPy: scipy.stats.chi2_contingency · docs.scipy.org Holm (1979): A Simple Sequentially Rejective Multiple Test Procedure, 6(2), 65–70
Hilfsmittel	OpenAI Codex / ChatGPT: Unterstützung bei Code, Dokumentation, Prüfung und Folien Python, Pandas, SciPy, Matplotlib, SQLite; Folien: python-pptx und PowerPoint

Vollständige Quellen, Methoden und Ausführung: README.md